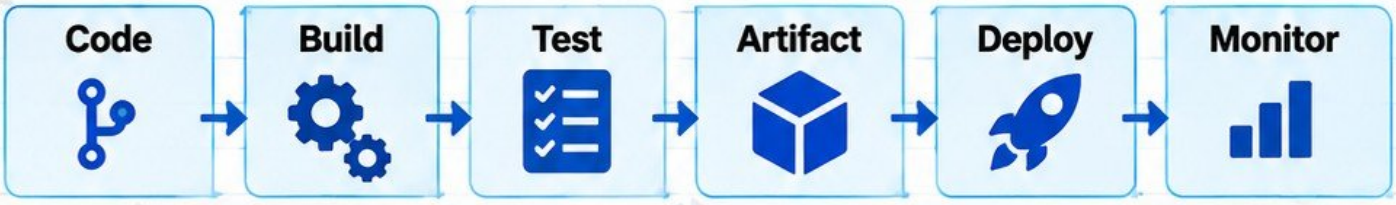
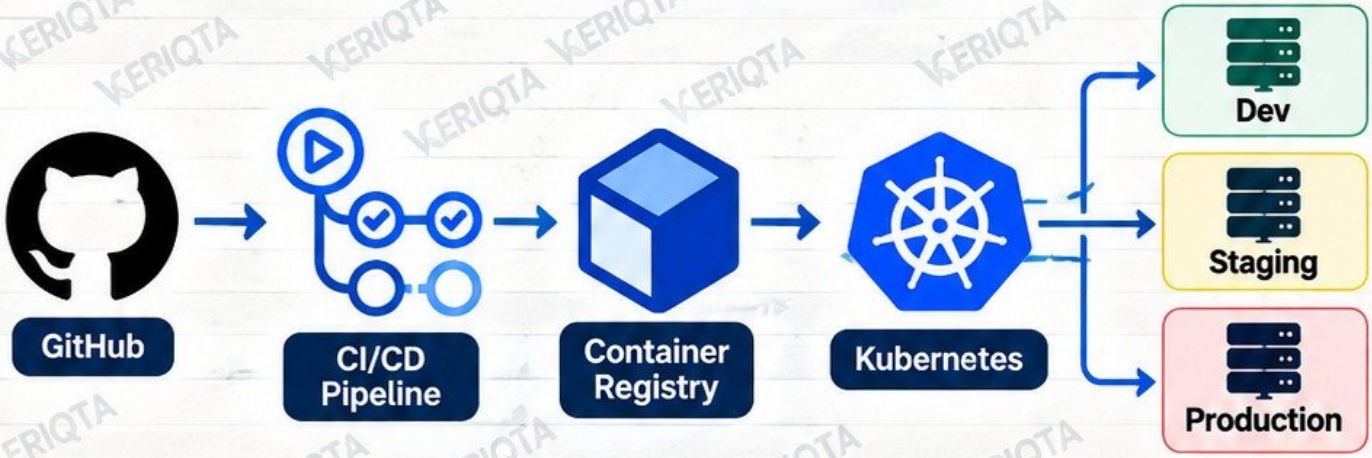
VERIQTA

CI/CD Operations Manual

Design | Operate | Secure | Scale | Troubleshoot | Recover







veriqta veriqta veriqta veriqta veriqta veriqta

CI/CD as a Production Control System

A CI/CD pipeline is not just a script that runs tests and deploys code. It is a production control system that moves changes from your codebase to real users. It controls what is released, how it is released and how risk is managed.

1. Pipeline versus Delivery System

Pipeline

- A set of automated jobs that build, test and deploy.
- Focuses on execution.
- Usually a tool configuration (GitHub Actions, GitLab CI, Jenkins etc).
- Can be green even if the release is unhealthy.

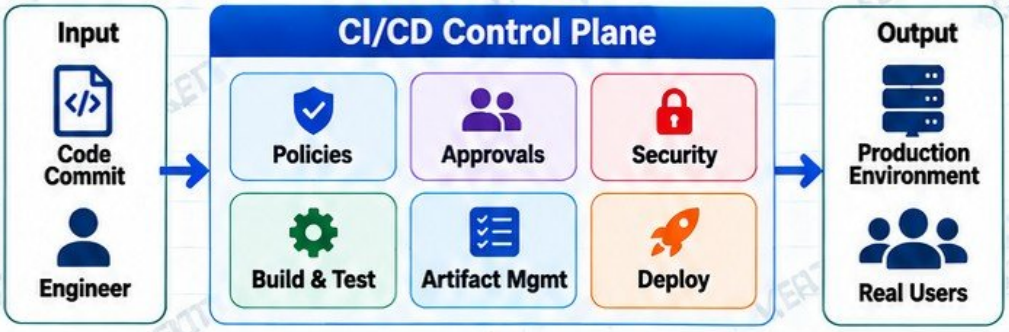
Delivery System

- End-to-end process, people, policies, tools and controls.
- Includes approvals, security, artifact management, environment promotion, observability and rollback.
- Focused on safe and reliable delivery to production.
- Designed to manage risk.

Key Difference

- Pipeline = automation layer.
- Delivery system = complete control system for production.
- Senior engineers design a delivery system, not just a pipeline.

2. Control Plane Thinking



The control plane decides:

- What can be built
- What can be deployed
- Where it can be deployed
- Who can approve
- What security checks run
- How to roll back
- How to monitor the release

3. Source to Production Flow



1. **Commit** — Code pushed to repository
2. **Build** — Build application and create artifact
3. **Test** — Run automated tests (unit, integration, security etc)
4. **Artifact** — Store immutable artifact in registry
5. **Deploy** — Deploy to target environment (dev, staging, prod)
6. **Monitor** — Verify release health and user impact

4. Trust Boundaries

Developer
Can commit code but should not have direct production access.

CI/CD System
Has limited, scoped access to cloud and cluster resources.

Production Environment
Only trusted, verified artifacts can be deployed.

Enforce least privilege
and use short-lived credentials.

5. Failure Domains

- **Source control failure** (cannot push or fetch code)
- **Build infrastructure failure** (runners, dependencies)
- **Artifact repository failure** (cannot store or retrieve)
- **Deployment failure** (cluster, cloud, configuration)
- **External dependency failure** (container registry, package repositories)
- **Production environment failure** (service unhealthy after deployment)

6. Why a Green Pipeline Does Not Prove a Healthy Production Release

Pipeline Succeeded

- Build passed
- Tests passed
- Artifact created
- Deployment completed

≠

Production Can Still Fail

- Application errors
- Misconfiguration
- Database issues
- Dependency failures
- Performance problems
- User-facing errors
- Feature not working
- Data issues

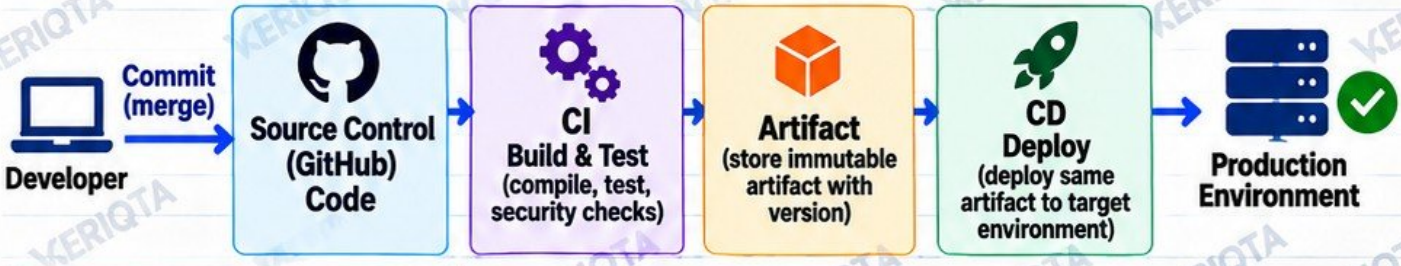
Key Point

A green pipeline only proves that the deployment process completed. You must still verify the health and behaviour of the application in production.

Designing the End-to-End Delivery Architecture

A well-designed delivery architecture moves a change from commit to production in a controlled, secure, observable and repeatable way. It defines the flow, environment boundaries, approval points and where failures should be contained.

1. Commit to Build to Artifact to Deployment



2. CI and CD Separation

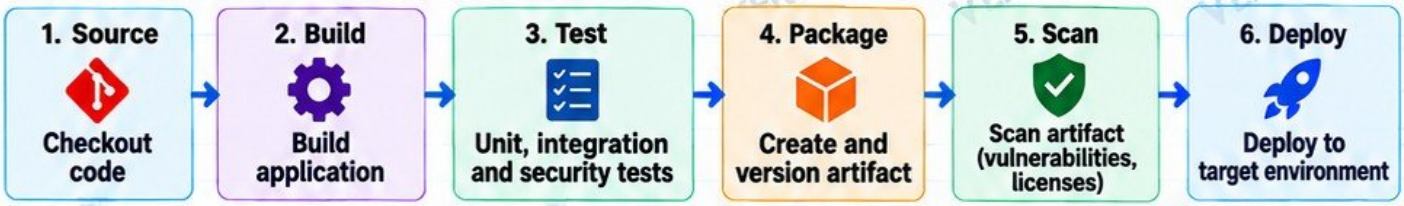
Continuous Integration (CI)

- Triggered on every commit.
- Builds the application.
- Runs automated tests.
- Performs security checks.
- Creates an immutable artifact.
- Does not deploy to production (usually).

Continuous Delivery/Deployment (CD)

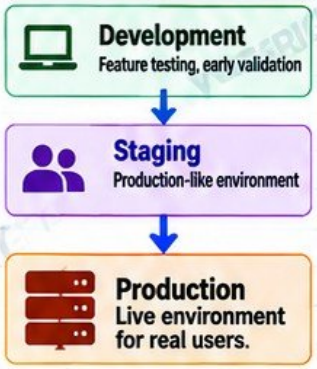
- Takes the tested artifact from CI.
- Deploys to environments (dev → staging → production).
- Runs additional validation.
- Can be manual approval (CD) or automatic (continuous deployment).
- Manages rollout, rollback and release control.
- Focuses on safe and reliable delivery.

3. Pipeline Stages and Dependencies



Dependencies Each stage depends on the success of the previous stage. If a stage fails, the pipeline stops and later stages do not run.

4. Environment Boundaries



- Separate infrastructure and credentials.
- Different access controls.
- Environment-specific configuration.
- No direct deployment to production from CI.

5. Control Points and Approvals

- **Code Review** (merge to main)
- **Automated Checks** (tests, security, compliance)
- **Manual Approval** (before production)
- **Policy Enforcement** (who can deploy, what can deploy)
- **Release Control** (scheduled or change window)

6. Where Failures Should Be Contained

- Build failures → contained in CI.
- Test failures → contained in CI.
- Security scan failures → contained before deployment.
- Configuration errors → contained in non-production environments.
- Deployment failures → contained in target environment (staging/prod).
- Production health issues → mitigated with rollout controls and fast rollback.

Build Once, Promote the Same Artifact

Build the application once, create an immutable artifact, and promote the same artifact through dev, staging and production. This ensures consistency, reduces risk and prevents environment drift.

1. Immutable Artifacts



- An immutable artifact cannot be changed after it is built.
- It is the exact same file or image promoted to all environments.
- Ensures consistency, traceability and reliability.
- Includes application code, dependencies and version.

2. Artifact Identity and Versioning

Artifact Identity Examples

Version number (SemVer)	v1.4.2
Git commit SHA	3f2a9c7
Build number	build-158
Container image tag	myapp:1.4.2
Artifact digest (immutable)	sha256:ab12...

3. Environment-Specific Configuration



- Keep the application artifact the same across all environments.
- Use separate configuration for dev, staging and production.
- Inject configuration at deploy time (environment variables, config files, secrets, or a configuration service).
- Allows the same artifact to behave differently per environment.

Example

```
# application.yaml
database:
  host: ${DB_HOST}
  port: ${DB_PORT}
featureFlag: ${FEATURE_FLAG}
logLevel: ${LOG_LEVEL}
```

4. Why Rebuilding per Environment Creates Drift



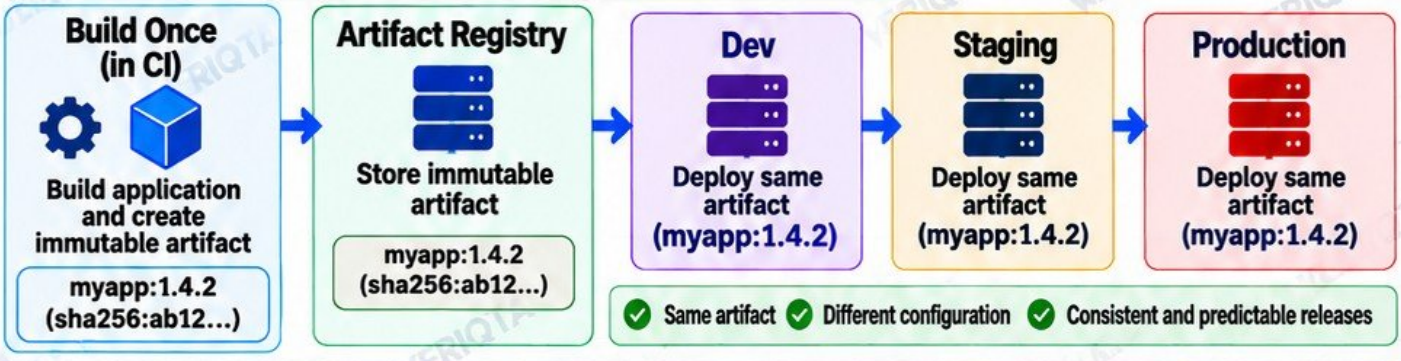
- Different builds can produce different binaries.
- Dependencies may change between builds.
- Environment differences can introduce unintended changes.
- Harder to debug and reproduce issues.
- Increases the risk of "works in staging but fails in production".

5. Provenance



- Provenance shows where the artifact came from and how it was built.
- Includes source code, build environment, dependencies and checksums.
- Helps with security, audit and supply-chain protection.
- Tools: SLSA, in-toto, Sigstore, Cosign, artifact attestation.

6. Promotion Through Dev, Staging and Production



Reproducible Builds and Dependency Control

A reproducible build always produces the same output from the same source and dependencies. This reduces risk, prevents environment drift and ensures consistency across dev, staging and production.

1. Deterministic Builds

- Same source code and dependencies always produce the same artifact.
- Eliminates "works on my machine".
- Improves reliability and traceability.
- Required for secure and auditable releases.

2. Dependency Pinning

- Use fixed versions instead of ranges.
- Prevents unexpected updates breaking your build.
- Reduces security and compatibility risks.
- Ensures consistent behavior across all environments.

3. Lockfiles

- Lockfiles record exact dependency versions.
- Ensures all environments install the same versions.
- Must be committed to version control.
- Examples: package-lock.json (Node.js), requirements.txt (Python), go.sum (Go), pom.xml (Maven).

Examples

Node.js — package-lock.json

```
{
 "name": "app",
 "version": "1.0.0",
 "dependencies": {
  "express": "4.18.2"
 }
}
```

Python — requirements.txt

```
flask==2.3.3
requests==2.31.0
pydantic==2.4.2
```

Go — go.sum

```
github.com/pkg
v1.2.3 h1:xxx
github.com/abc
v0.4.0 h1:yyy
```

Java — pom.xml

```
<dependency>
  <groupId>org</groupId>
  <artifactId>spring</ar-
  rtifactId>
  <version>3.1.2</version>
</dependency>
```

4. Build Environment Consistency

- Use containers or defined build images.
- Keep the same OS, runtime, and tools across all environments.
- Version your build environment (Dockerfile, pinned tool versions).
- Avoid using local machine configuration for builds.

Example: Docker-based Build Environment

```
FROM node:20.10.0
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
```

- Ensures the same Node.js version.
- Uses lockfile for exact dependencies (npm ci).
- Produces consistent builds across all environments.

5. Cache Risks

- Caches can speed up builds but introduce risks.
- Stale or corrupted caches can break builds.
- Caches may hide missing dependencies.
- Cache keys must include dependency versions.
- Clear cache when facing unexplained build errors.

6. External Dependency Failures

- External package registries can be slow, unavailable or rate-limited.
- Can cause CI builds to fail even if your code is correct.
- Use retries, timeouts and mirrors (e.g. private registry).
- Consider caching dependencies in your organization.

7. Diagnosing "Works Locally, Fails in CI"

1. **Compare Environments** — Check OS, runtime, tool versions and environment variables.
2. **Check Dependencies** — Ensure lockfile is used and all versions match.
3. **Review Build Logs** — Look for version mismatches, missing packages or network errors.
4. **Check External Services** — Verify package registries and other dependencies are reachable.
5. **Reproduce in a Clean Environment** — Use a clean container or CI-like environment locally.

Pipeline Isolation and Runner Architecture

Runners execute your CI/CD jobs. The way you design and isolate runners directly impacts security, reliability, performance and cost. Choose the right runner architecture based on your organization's risk, scale and compliance requirements.

1. Hosted vs Self-Hosted Runners

Hosted Runners

- Managed by the provider (GitHub, GitLab, etc).
- No infrastructure to manage.
- Scales automatically.
- Suitable for most use cases.
- Limited customization.
- Shared with other customers (multi-tenant).

Self-Hosted Runners

- Run in your own infrastructure (VM, Kubernates, on-prem).
- Full control and customization.
- Access to private resources (VPC, internal systems).
- Required for compliance or restricted environments.
- You are responsible for scaling, maintenance and security.

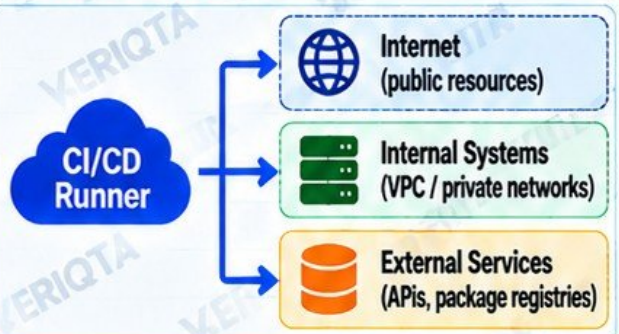
2. Ephemeral Workers

- Created for a single job.
- Destroyed after the job completes.
- Provides clean and isolated environment.
- Prevents workspace contamination.
- Reduces security risk.
- Commonly implemented using containers or auto-scaling VMs.

3. Shared-Runner Risks

- Multi-tenant environment.
- Potential exposure to malicious code.
- Resource contention (CPU, memory, disk).
- Slower builds due to queue and load.
- Limited control over installed tools.
- Not suitable for sensitive or regulated workloads.

4. Network Access



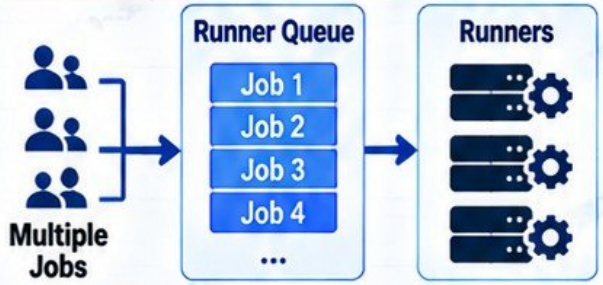
5. Privilege Boundaries

- Run with least privilege.
- Avoid using root/admin access.
- Restrict access to only required resources.
- Use separate runners for sensitive jobs (e.g. deployments).
- Use cloud IAM roles or workload identity instead of long-lived credentials.

6. Workspace Contamination

- In shared or persistent runners, files and tools can remain from previous jobs.
- Can cause unexpected build behavior.
- May expose secrets or sensitive files.
- Use ephemeral runners or clean workspace between jobs.
- Always verify what is cached or persisted in the runner.

7. Capacity and Queue Saturation



Common Causes

- Too many concurrent jobs.
- Insufficient runner capacity.
- Long-running jobs.
- Resource-heavy builds.
- Large monorepos.
- Unoptimized test suites.

Solution

- Use auto-scaling runners.
- Set concurrency limits.
- Optimize build and test times.
- Use job prioritization.
- Split heavy jobs.
- Monitor runner queue and execution time.

Secrets, Identity, and Pipeline Credentials

Secure, short-lived and least-privilege credentials keep your CI/CD pipeline safe. Avoid long-lived static secrets and use workload identity with OIDC to access cloud resources securely.

1. Static Secrets vs Workload Identity

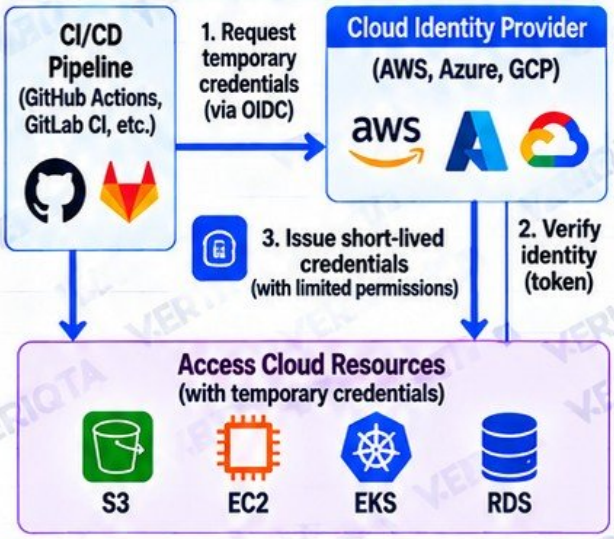
Static Secrets

- Long-lived credentials.
- Manually created and stored.
- Higher risk of leakage.
- Harder to rotate.
- Often over-privileged.
- Commonly stored in CI/CD secret variables.

Workload Identity (OIDC)

- Short-lived, automatically issued credentials.
- No long-lived secrets stored in the pipeline.
- Uses OIDC federation.
- Scoped permissions (least privilege).
- Rotates automatically.
- More secure and auditable.

2. How OIDC Federation Works



3. Short-Lived Credentials

- Automatically issued with a limited time (e.g. 15 minutes).
- Reduce the impact if a credential is exposed.
- No manual rotation required.
- Used with OIDC, STS, or cloud workload identity.
- Access only the resources needed for the job.

4. Least Privilege

- Grant only the permissions required for the pipeline job.
- Use fine-grained IAM roles or service accounts.
- Separate roles for build, test and deploy.
- Avoid using admin or broad permissions.
- Review permissions regularly.

5. Secret Exposure Paths

- Hard-coded secrets in code or scripts.
- Exposed in build logs.
- Stored in environment variables.
- Leaked through artifacts or container images.
- Shared between teams without access control.
- Overly permissive IAM roles.
- Secrets left in self-hosted runners.

6. Rotation Strategies

- Use short-lived credentials (preferred).
- Rotate static secrets regularly.
- Automate rotation using a secrets manager (e.g. AWS Secrets Manager, Azure Key Vault).
- Revoke old credentials immediately.
- Monitor and alert on unusual secret usage.

7. Preventing CI from Becoming an Infrastructure Master Key

- Use workload identity instead of storing cloud keys.
- Enforce least privilege for all pipeline jobs.
- Separate roles for different environments.
- Restrict network access from runners.
- Use approvals for production deployments.
- Audit and monitor pipeline access.
- Do not allow pipelines to have full admin access.

8. Example: GitHub Actions with AWS OIDC (No Static Secrets)

```yaml
name: Deploy to AWS
permissions:
  id-token: write   # Required for OIDC
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
          aws-region: us-east-1
      - name: Deploy
        run: echo "Deploying to AWS..."
```

What This Does

- Uses OIDC to authenticate GitHub to AWS.
- No static AWS keys stored in the pipeline.
- Requests short-lived credentials.
- Assumes a specific IAM role with limited permissions.
- Credentials automatically expire.
- Reduces the risk of credential leakage.

Key Takeaways

- Prefer OIDC and workload identity.
- Use short-lived credentials.
- Apply least privilege.
- Rotate and monitor credentials.
- Keep pipelines secure and isolated.

CI/CD Supply-Chain Security

Secure the entire software supply chain from source to runtime. Verify what you use, find risks early, and ensure artifacts are trusted, traceable and tamper-proof.

1. Dependency Trust

- Use trusted and official sources (package registries).
- Verify maintainer reputation.
- Check for known vulnerabilities.
- Pin dependency versions.
- Avoid unmaintained or abandoned packages.
- Use internal or private registries for critical dependencies.

2. Build Provenance

- Track how the artifact was built (source, workflow, environment).
- Know who built it, when and with what.
- Use provenance attestation (e.g. SLSA).
- Helps detect tampering and unauthorized changes.
- Provides traceability from source to runtime.
- Required for regulated and high-security environments.

3. SBOMs (Software Bill of Materials)



- Lists all components, libraries and dependencies.
- Helps identify vulnerable components.
- Used for security, compliance and audit.
- Generate SBOMs automatically (e.g. Syft, CycloneDX, SPDX).
- Keep SBOMs with your artifacts.

4. Artifact Signing

- Digitally sign artifacts (images, binaries, packages).
- Verifies integrity and authenticity.
- Allows you to detect tampering.
- Use tools like Cosign, Notary, or GPG.
- Required for trusted promotion to production.

5. Image Scanning

- Scan container images for vulnerabilities.
- Check both OS packages and application dependencies.
- Use tools like Trivy, Grype or ECR/GAR scanners.
- Fail the pipeline on critical or high vulnerabilities.
- Scan base images and custom images.

6. Pipeline Dependency Risk

- Your pipeline uses actions, plugins and third-party tools.
- These can be compromised or unmaintained.
- Use trusted and version-pinned actions.
- Review permissions for CI tools.
- Monitor for updates and security advisories.
- Limit what third-party tools can access.

7. Protecting the Path from Source to Runtime

1. Source Code	2. Dependencies	3. Build	4. Artifact	5. Deployment	6. Runtime
Secure repository (branch protection, code review)	Use trusted sources, pin versions, generate SBOM	Reproducible build, provenance attestation	Signed artifact, scanned for vulnerabilities	Verify signature, use trusted registry, enforce policies	Run hardened, monitored and trusted artifacts

Security Controls

- Trusted dependencies
- Pinned versions
- SBOM generation
- Artifact signing
- Image scanning
- Provenance attestation
- Least privilege
- Policy enforcement
- Continuous monitoring
- Audit and compliance

Tools

- SBOM: Syft, CycloneDX
- Image scanning: Trivy, Grype
- Signing: Cosign, Notary
- Provenance: SLSA, in-toto
- Registries: ECR, GAR, ACR
- Policy: OPA, Kyverno, Conftest

Key Takeaways

- Verify everything you use.
- Automate security in the pipeline.
- Ensure artifacts are traceable and signed.
- Detect vulnerabilities before production.
- Control third-party and pipeline risks.
- Protect the path from source to runtime.

Branch Protection and Change Governance

Branch protection and change governance ensure that the right changes are made, reviewed and verified before reaching production. This reduces risk, maintains code quality and balances delivery speed with the stability of your production environment.

1. Protected Branches

- Protect critical branches (e.g. main, release, production).
- Prevent direct pushes.
- Require pull requests for changes.
- Enforce required checks and reviews.
- Limit who can bypass protection.
- Keeps production code stable and controlled.

Example: GitHub Branch Protection

```
{
  "required_status_checks": {
    "strict": true,
    "contexts": ["build", "test", "security"]
  },
  "enforce_admins": true,
  "required_pull_request_reviews": {
    "required_approving_review_count": 2
  },
  "restrictions": {
    "users": ["dev1", "dev2"]
    "teams": ["platform-team"]
})
```

2. Required Checks

- Require CI checks to pass (e.g. build, test, security scan).
- Prevent merge if any check fails.
- Ensure code meets quality and security standards.
- Common checks: unit tests, integration tests, linting, image scanning, policy checks.

3. CODEOWNERS

- Define who must review specific files or directories.
- Automatically request reviews from code owners.
- Ensures domain experts review critical changes.
- Helps maintain ownership and accountability.

Example: CODEOVNERS File

```
# Global owners
*       @platform-team

# Infrastructure
/infrastructure/  @infrastructure-team

# Kubernetes manifests
/k8s/    @kubernetes-team

# Application code
/src/    @app-team

# Security configs
/security/  @security-team
```

4. Review Controls

- Require a minimum number of approvals (e.g. 2 reviewers).
- Use automated and manual reviews.
- Review for security, performance, and production impact.
- Block merges if reviews are not approved.
- Encourage meaningful code reviews, not just approvals.

5. Separation of Duties

- Different people create, review and approve changes.
- Prevents a single person from pushing unverified changes to production.
- Use teams, roles and permissions to enforce separation.

6. Emergency Changes

- Have a clear process for urgent production fixes.
- Use time-limited approvals.
- Still require basic checks (e.g. build and security scan).
- Log and audit all emergency changes.
- Review emergency changes after deployment.

7. Balancing Delivery Speed Against Production Risk

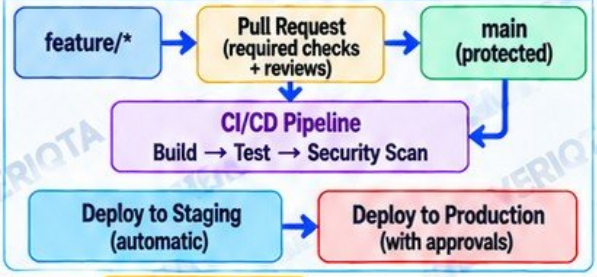
Delivery Speed

- Faster feature delivery
- Less waiting time
- Higher developer productivity
- Supports business needs

Production Risk

- Higher chance of bugs
- Security vulnerabilities
- Unstable releases
- Increased rollback risk
- Potential customer impact

Typical Branch Strategy



Find the Right Balance

- Use automation to speed up checks.
- Apply stricter controls for production branches.
- Use progressive delivery (e.g. canary, feature flags).
- Continuously monitor and improve the process.

Key Takeaways

- Protect critical branches and require checks.
- Use CODEOWNERS and review controls.
- Separate duties to reduce risk.
- Have a clear process for emergency changes.
- Balance speed and safety with automation and progressive delivery.

Pipeline Concurrency, Queues, and Resource Contention

As your team and codebase grow, multiple pipeline runs compete for runners, resources and environments. Understanding concurrency, queues and resource contention helps you keep your CI/CD fast, reliable and cost-efficient.

1. Parallel Execution



- Run independent jobs in parallel to reduce pipeline time.
- Use a job matrix for multiple versions or environments.
- Avoid unnecessary sequential dependencies.
- Ensure shared resources (e.g. caches, environments) are not conflicted.

2. Runner Capacity



- Runners execute your jobs.
- Capacity depends on the number of runners and their specifications (CPU, memory, disk).
- Insufficient runners cause queues and longer wait times.
- Use auto-scaling runners to adjust to demand.
- Monitor runner utilization and cost.

3. Queue Latency



- Jobs wait in a queue when no runner is available.
- Queue latency increases as demand grows.
- Long queues delay feedback and slow down delivery.
- Monitor queue time and set alerts.
- Scale runners or optimize jobs to reduce latency.

4. Concurrency Limits



- Set limits on how many pipelines or jobs can run at the same time.
- Prevent overloading runners, environments or external services.
- Use concurrency controls per branch, environment or job type.
- Helps maintain stability and fair resource usage.

5. Duplicate Builds



- Multiple commits or pull requests can trigger duplicate pipelines.
- Wastes runner resources and increases cost.
- Can cause conflicting deployments or race conditions.
- Use conditional triggers and cooldown periods to reduce duplicates.

6. Cancellation Strategies



- Cancel outdated pipeline runs when a new commit is pushed.
- Use auto-cancellation for superseded commits or pull requests.
- Save runner resources and reduce cost.
- Ensure only the latest relevant pipeline runs to completion.

7. What Happens When Hundreds of Engineers Push Simultaneously?



Common Challenges



- Long queue times
- Runner resource exhaustion
- Slower feedback for developers
- Increased cost
- Potential failed or canceled jobs
- Impact on downstream environments

Solutions



- Use auto-scaling runners
- Optimize job parallelization
- Set concurrency limits
- Cancel redundant runs
- Monitor queue times and runner usage
- Use caching to reduce job time

Key Takeaways



- Parallel execution speeds up delivery.
- Sufficient runner capacity is critical.
- Queue latency impacts developer productivity.
- Use concurrency limits and cancellation strategies.
- Plan for high-traffic events.
- Monitor and optimize continuously.

Deployment Strategies and Their Failure Modes

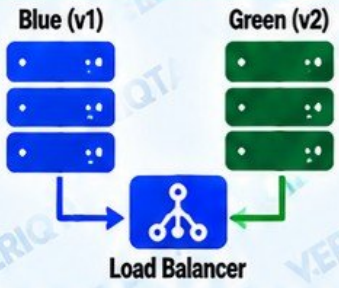
Choose the right deployment strategy based on your application's requirements, rollback needs, blast radius and cost. Each strategy has different trade-offs, failure modes and operational considerations.

1. Rolling Deployment



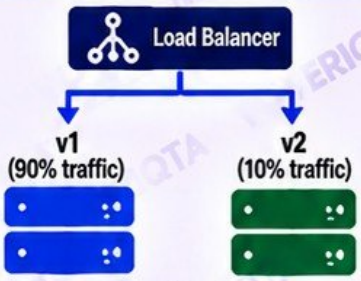
- Replace instances gradually (one by one or in batches).
- No downtime.
- Traffic is served by a mix of old and new versions.
- **Failure Mode:** new version has a bug, causing errors while rollout is in progress.
- **Rollback:** stop rollout and roll back to previous version.

2. Blue-Green Deployment



- Two identical environments (blue and green).
- Switch all traffic from blue to green.
- Instant cutover, minimal downtime.
- **Failure Mode:** issues after cutover require switching back to blue.
- **Rollback:** quickly switch traffic back to blue.

3. Canary Deployment



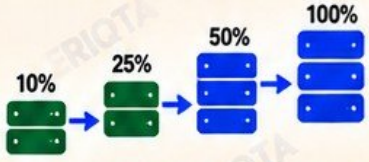
- Release to a small percentage of users first (canary).
- Monitor metrics, logs and errors.
- Gradually increase traffic if healthy.
- **Failure Mode:** issues affect a small subset of users.
- **Rollback:** stop traffic to canary version.

4. Recreate Deployment



- Stop all running instances and start new ones.
- Simple and fast.
- Causes downtime.
- **Failure Mode:** service is unavailable during replacement.
- **Rollback:** redeploy previous version.

5. Progressive Delivery



- Gradually increase traffic to new version based on monitoring and success criteria.
- Can be feature-flag driven.
- Reduces risk by detecting issues early.
- **Failure Mode:** issues during later stages can still impact more users.
- **Rollback:** stop at previous stable stage or disable feature flag.

6. Capacity and Cost Implications



- **Rolling:** temporary extra capacity may be needed during rollout.
- **Blue-Green:** requires double infrastructure (runs two full environments).
- **Canary:** lower extra cost (only a small percentage of traffic).
- **Recreate:** no extra capacity, but downtime can be costly.
- **Progressive:** moderate extra capacity, cost depends on rollout stages.

7. Comparison of Deployment Strategies

Strategy	Downtime	Rollback Speed	Blast Radius	Infrastructure Cost	Best Use Case
Rolling	No	Medium	Medium	Medium	General applications
Blue-Green	Minimal	Fast	Low	High	Critical applications
Canary	No	Fast	Low	Low–Medium	New features, risk reduction
Recreate	Yes	Fast	High	Low	Non-critical applications
Progressive	No	Fast	Low–Medium	Medium	Complex applications, feature rollout

8. Choosing the Right Strategy

Key Questions

- How much downtime can you tolerate?
- How fast do you need to roll back?
- What is the acceptable blast radius?
- What is your budget for extra capacity?
- Do you need to roll out features gradually?

Choose Strategy

- Low risk, zero downtime → Blue-Green / Canary
- Simple and low cost → Rolling
- Allow downtime → Recreate
- Feature rollout with control → Progressive

Key Takeaways

- No single strategy is best for every application.
- Consider rollback requirements and blast radius.
- Balance deployment speed, risk and cost.
- Use monitoring and automation to detect and respond to failures.
- Test your deployment strategy regularly.

Kubernetes Deployment Behavior Through CI/CD

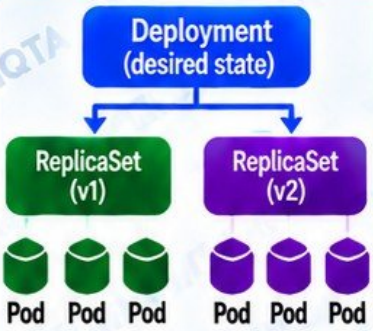
A Kubernetes deployment in CI/CD is more than just applying a manifest. Kubernetes reconciles the desired state, creates ReplicaSets, and performs a rolling update while checking pod readiness. Understanding this behavior helps you design safer and more reliable releases.

1. Deployment Reconciliation



- You declare the desired state (e.g. 3 replicas, image version).
- Kubernetes continuously reconciles the actual state to match the desired state.
- If a pod fails, Kubernetes creates a new one automatically.
- Reconciliation runs continuously, not just during deployment.

2. ReplicaSets



- A Deployment creates and manages ReplicaSets.
- Each ReplicaSet manages a specific version of pods (e.g. v1, v2).
- During an update, a new ReplicaSet is created.
- Old ReplicaSets are scaled down after a successful rollout.

3. RollingUpdate Strategy



- Replaces old pods gradually with new pods.
- Ensures zero or low downtime.
- Traffic is served by a mix of old and new pods during rollout.
- Requires readiness probes to avoid sending traffic to unhealthy pods.

4. maxSurge and maxUnavailable

- maxSurge: maximum extra pods above the desired replicas during rollout.
- maxUnavailable: maximum number of pods that can be unavailable during rollout.

Example: 3 replicas	
maxSurge: 1 maxUnavailable: 0	maxSurge: 0 maxUnavailable: 1
Up to 4 pods (3 + 1 surge)	One pod can be taken down at a time
No downtime (all 3 always available)	Slower rollout but no extra cost

5. Readiness Gates



- Readiness probes (and readiness gates) ensure a pod is actually ready before receiving traffic.
- A pod must pass all readiness checks (application, dependencies, custom conditions).
- If readiness fails, the pod is not added to service endpoints.
- This prevents sending traffic to unhealthy pods during rollout.

6. rollout status

```
# Check rollout status
kubectl rollout status deployment/myapp

# Example output
Waiting for deployment "myapp" rollout to finish:
2 out of 3 new replicas have been updated...
3 out of 3 new replicas have been updated...
deployment "myapp" successfully rolled out

# Check rollout history
kubectl rollout history deployment/myapp

# Rollback if needed
kubectl rollout undo deployment/myapp
```

7. Why "kubectl apply succeeded" Does Not Mean the Release Succeeded



- kubectl apply only validates and saves the desired state in Kubernetes.
- It does NOT wait for pods to be ready.
- The deployment can still fail after apply due to:
  - Image pull errors
  - Readiness probe failures
  - Crash loops
  - Application errors
  - Appendency or configuration issues
- Always check rollout status, pod health, logs and metrics to confirm the release actually succeeded.

Key Takeaways



- Kubernetes continuously reconciles the desired state.
- Deployments create and manage ReplicaSets.
- RollingUpdate replaces pods gradually.
- Use maxSurge and maxUnavailable to control rollout.
- Readiness gates prevent traffic to unhealthy pods.
- Always verify with kubectl rollout status.
- kubectl apply succeeded only means the manifest was accepted, not that the release is successful.

Database Changes Without Breaking Deployments

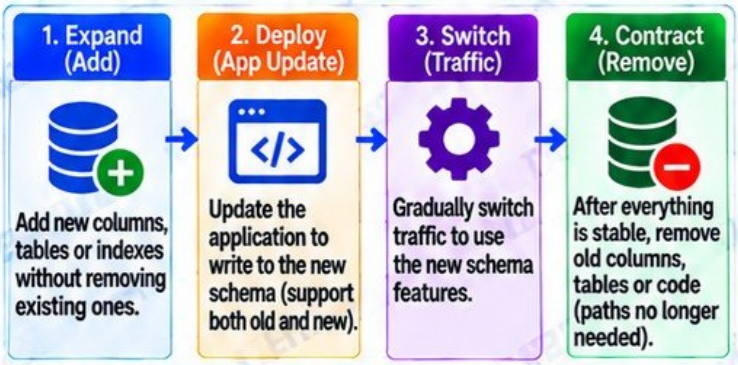
Database changes can break deployments if the application and schema are not compatible. Use safe migration strategies, maintain backward compatibility, and understand the risks, limitations and rollback implications.

1. Application and Schema Compatibility



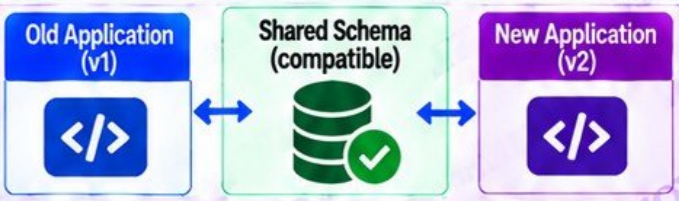
- The application version must be compatible with the database schema.
- Avoid breaking changes that cause runtime errors.
- Ensure both old and new application versions can work during the transition.
- Test schema changes with real application behavior (not just SQL).

2. Expand-Contract Migrations



This approach keeps the system working during the entire migration and allows safe rollbacks.

3. Backward Compatibility



- Make schema changes in a backward compatible way.
- Old and new application versions should be able to read and write without errors.
- Avoid removing columns or changing data types in a way that breaks older versions.
- Use feature flags to gradually enable new database features.

4. Long-Running Migrations



- Large tables can take a long time to migrate.
- These migrations can impact application performance if not planned properly.
- Use online migration tools or techniques that avoid full table locks.
- Run migrations in smaller batches.
- Monitor database performance (CPU, I/O, locks, replication lag) during the migration.

5. Locking



- Some schema changes can lock tables, blocking reads and writes.
- Table locks can cause high latency or outages.
- Use online DDL or non-blocking migration methods where possible.
- Always test migrations in a staging environment with production-like data.

6. Rollback Limitations



- Some schema changes cannot be easily rolled back (e.g. dropped columns, data transformations).
- Rolling back the application does not automatically revert database changes.
- You may need a separate rollback script or backup.
- Plan rollback strategy before running migrations.

7. Why Application Rollback Cannot Always Reverse a Release



This can cause:

- The old application may not work with the new schema.
- Missing columns, changed data types or removed tables can cause errors.
- Some data changes cannot be undone.
- You may need a separate rollback plan (e.g. reverse migration script or restore from backup).

Key Takeaways



- Ensure application and schema compatibility.
- Use expand-contract migrations.
- Maintain backward compatibility.
- Plan for long-running migrations and avoid locking.
- Understand rollback limitations.
- Test migrations and rollbacks.
- Application rollback does not always reverse database changes.

Configuration and Feature-Flag Delivery

Use configuration management and feature flags to deliver changes safely, separate deployment from feature release, and reduce risk in production while keeping full control over your application behaviour.

1. Code Versus Configuration

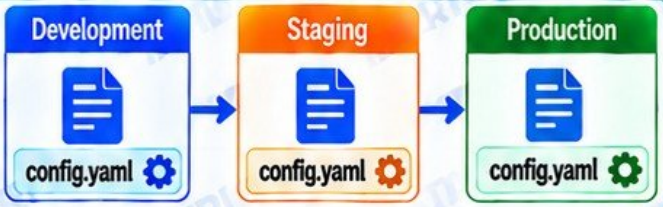
Code

- Defines application logic and functionality.
- Requires build, test and deployment.
- Changes usually need a new version.
- Example: new feature, bug fix, API changes.

Configuration

- Controls how the application behaves.
- Can often be changed without rebuilding code.
- Managed per environment (dev, staging, production).
- Example: database URL, log level, feature flag values, API endpoints.

2. Configuration Promotion



- Use version-controlled configuration (e.g. Git).
- Promote the same configuration through environments.
- Keep environment-specific values separate (e.g. dev/staging/prod).
- Use infrastructure as code and configuration management tools (e.g. Helm, Kustomize, Terraform, Ansible).
- Validate configuration before promotion.

3. Feature Flags



- Allow you to deploy code but control feature availability at runtime.
- Enable gradual rollout to a small percentage of users.
- Support A/B testing and quick rollback without redeployment.
- Reduce risk by keeping new features off until they are ready.
- Can be user-based, percentage-based, region-based or environment-based.

4. Runtime Changes



- Some configuration changes can be applied at runtime without redeploying the application.
- Examples: feature flag values, log levels, rate limits, external service endpoints.
- Use dynamic configuration systems (e.g. AWS AppConfig, Consul, ConfigMaps).
- Ensure changes are validated and monitored.
- Not all changes can be applied at runtime.

5. Configuration Drift



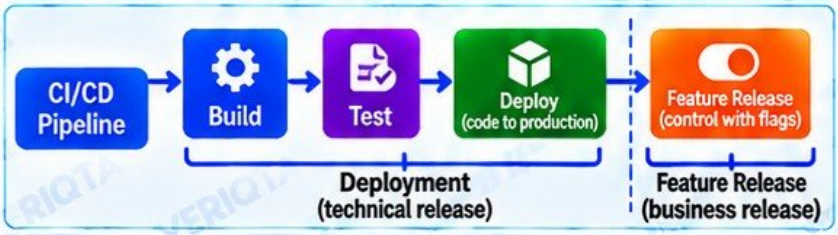
- Configuration drift happens when environments become inconsistent.
- Can be caused by manual changes, hotfixes or untracked updates.
- Leads to unpredictable behaviour and deployment issues.
- Prevent drift using version control, infrastructure as code and automated configuration management.
- Regularly audit and compare configurations across environments.

6. Kill Switches



- A kill switch quickly disables a feature or functionality in production.
- Helps stop incidents and reduce blast radius.
- Can be implemented using feature flags or configuration changes.
- Should be easy to use, well tested and accessible to the on-call team.
- Use for high-risk features, third-party integrations or during active incidents.

7. Separating Deployment from Feature Release



Key Takeaways



- Understand the difference between code and configuration.
- Use version control and proper promotion for configuration.
- Use feature flags to control feature release.
- Be aware of configuration drift and prevent it.
- Implement kill switches for high-risk changes.
- Separate deployment from feature release to reduce risk and increase flexibility.

Release Health and Deployment Observability

A release is not successful just because the deployment completed. You need data to confirm that the new version is actually healthy in production. Monitoring key metrics, logs and traces helps you detect issues early and make informed decisions.

1. Deployment Metrics

- Track key metrics to understand how the new version is performing.
- Compare metrics before and after the release.
- Look at application metrics, infra metrics and business metrics.

Common Deployment Metrics

- **Request rate:** number of requests per second.
- **Error rate:** percentage of failed requests.
- **Latency:** time taken to respond to requests.
- **Saturation:** CPU, memory, disk, network usage.
- **SLO burn rate:** how quickly you are using your error budget.

2. Error Rate

- Monitor the percentage of failed requests (4xx, 5xx).
- Compare error rate before and after the release.
- A sudden increase may indicate a problem with the new version.

Example: Error Rate



3. Latency

- Track response time (e.g. p50, p95, p99).
- Higher latency can indicate performance issues, resource bottlenecks or inefficient code.
- Compare latency before and after the release.

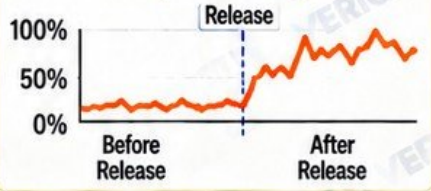
Example: Latency (p95)



4. Saturation

- Check resource usage to see if the new version is putting extra load on the system.
- Monitor CPU, memory, disk, network and any external dependencies (databases, queues, third-party APIs).
- High saturation can lead to slowdowns, errors or outages.

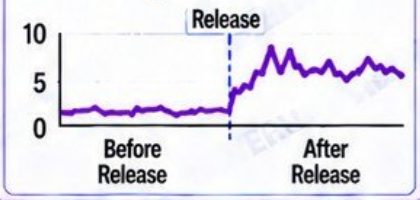
Example: CPU Usage



5. SLO Burn Rate

- SLO burn rate shows how fast you are using your error budget.
- A high burn rate means the release may be causing more errors than expected.
- Monitor short-term and long-term burn rates (e.g. 1 hour and 6 hours).

Example: SLO Burn Rate



6. Logs and Traces

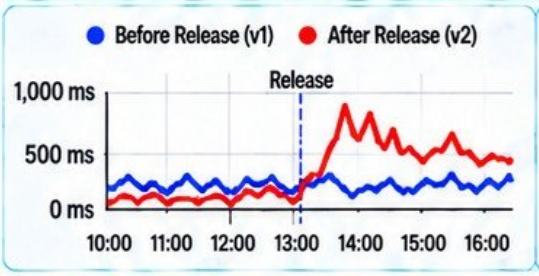
- Use logs to find application errors, exceptions and unusual behaviour.
- Use distributed traces to identify slow requests and failing dependencies.
- Compare logs and traces before and after the release to spot new issues.

Example: Traces



Traces help you see exactly where latency or errors are happening.

7. Comparing Pre-Release and Post-Release Behaviour



- Compare key metrics (error rate, latency, resource usage) before and after the release.
- Look for significant changes or upward trends.
- Start monitoring immediately after the release and continue for a sufficient period.

8. Defining Evidence for a Healthy Release

- Error rate remains at or below expected levels.
- Latency is stable and within SLO targets.
- Resource usage is normal and not increasing unexpectedly.
- No critical errors in logs or traces.
- SLO burn rate is within acceptable limits.
- Business metrics (e.g. user signups, purchases) are healthy.
- No increase in customer complaints or alerts.

Key Takeaways

- Monitor deployment metrics, not just deployment status.
- Compare pre-release and post-release behaviour.
- Use logs and traces to find the root cause of issues.
- Watch SLO burn rate to understand impact on reliability.
- Define clear criteria for a healthy release.
- Set up alerts for key metrics.
- A successful deployment means the new version is running well and meeting performance, reliability and business goals.

The Deployment Succeeded, Production Failed

A successful pipeline only means the code was built, tested and deployed. It does not guarantee that the service is healthy in productiion. Many issues appear only after the deployment completes.

1. Successful Pipeline, Unhealthy Service



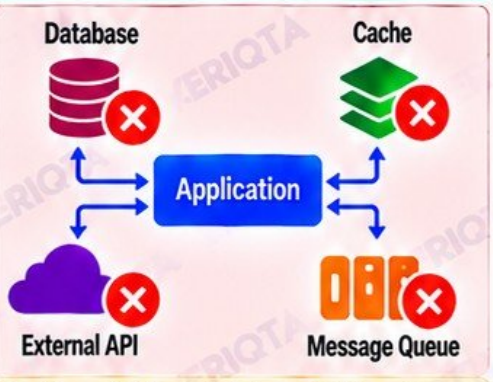
- The pipeline completed successfully (build, test, deploy).
- The new version is running in production.
- But users experience errors, high latency or downtime.
- Deployment success does not always mean the service is healthy.

2. Readiness Mistakes



- Readiness probes may be too simple or misconfigured.
- The application may report ready before it can actually serve traffic.
- Dependencies (database, cache, external services) may not be ready.
- Traffic is sent to the pod, but requests fail.

3. Dependency Failures



- The application starts, but a required dependency is down or slow.
- Examples: database connection failures, cache unavailable, third-party API errors, message queue issues.
- The service may be "up" but still return errors to users.

4. Configuration Errors

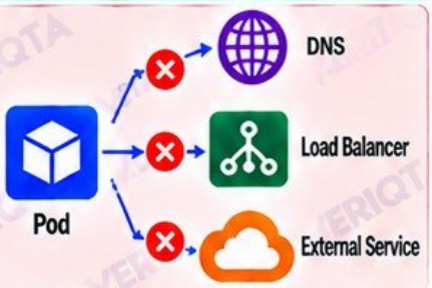


Production Config

- Wrong value
- Missing value
- Old value
- Environment mismatch
- Invalid format

- Configuration mistakes can cause runtime failures.
- Examples: wrong environment variables, incorrect feature flag values, wrong database URL, bad service endpoints.
- The application may start successfully but behave incorrectly.

5. DNS and Network Failures



- DNS records may be incorrect or not propagated yet.
- Network policies, firewalls or security groups may block traffic.
- Load balancers may be misconfigured.
- The application may be healthy, but cannot reach required services.

6. Partial Failures



- Only a subset of instances may be failing.
- Some users experience errors, while others are unaffected.
- Can be caused by pod scheduling issues, data inconsistency, configuration drift or dependency timeouts.
- Partial failures can be hard to detect without proper monitoring.

7. Establishing Blast Radius Before Changing Production

1. Identify what is affected (users, regions, endpoints).
2. Check the scope (is it all traffic or a subset?).
3. Look at metrics, logs and traces to understand the impact.
4. Determine if the issue is isolated or widespread.
5. Establish the blast radius before making any changes.

Key Questions

- Which users are affected?
- Is it one region or global?
- Is it one endpoint or the entire service?
- Is the issue new or existing?
- What changed just before the problem started?
- Can we mitigate without increasing the blast radius?

Key Takeaways

- A successful deployment does not guarantee a healthy service.
- Check readiness, dependencies, configuration, DNS and network.
- Partial failures can affect only some users.
- Use metrics, logs and traces to confirm the real impact.
- Establish the blast radius before making changes.
- Monitor closely after every deployment.

Failed Pipeline Investigation

A failed pipeline can be caused by many things: code changes, infrastructure issues, dependency failures or flaky tests. A structured investigation helps you find the real cause quickly and avoid wasting time on repeated reruns.

1. Establish the Failing Stage



- Identify exactly which stage failed.
- Check the stage logs and error message.
- Determine if it failed consistently or only once.
- Look at recent commits, configuration changes or dependency updates.
- Understand what the stage is supposed to do.

2. Separate Infrastructure Failure from Application Failure

Infrastructure Failure (environment issue)

- Runner unavailable or out of capacity.
- Network, DNS or artifact repository issues.
- Cloud provider or platform outage.
- Permission or authentication errors.
- Timeouts when downloading dependencies.

Application Failure (code or application issue)

- Compilation or build errors.
- Unit or integration test failures.
- Application packaging errors.
- Configuration or environment variable issues.
- Application-specific dependency failures.

3. Recent-Change Correlation

- Check recent code changes (commits, pull requests).
- Look at dependency updates (libraries, base images).
- Review configuration changes (pipeline file, environment variables).
- Check for infrastructure changes (runner, network, permissions).
- Compare with the last successful run to identify differences.

4. Runner Evidence

- Check runner status (online, healthy, not deleted).
- Look at runner logs for errors.
- Verify runner disk space, memory and CPU.
- Confirm network access to external services (e.g. package registries).
- Ensure required tools and versions are installed.

5. Build Logs

```
1 [INFO] Starting build...
2 [INFO] Downloading dependencies...
3 [ERROR] Failed to download package
4 [ERROR] Connection timed out
5 [INFO] Retrying (1/3)...
6 [ERROR] Connection timed out
7 [ERROR] Build failed after 3 retries
```

- Read the full logs, not just the last few lines.
- Look for the first error message (root cause).
- Identify patterns (timeouts, permission errors, missing files).
- Check if the error is related to code, dependencies or infrastructure.

6. Dependency Failures

- External dependencies (package registries, container registries, APIs) may be unavailable.
- Version changes can introduce breaking changes.
- Network issues can prevent downloading dependencies.
- Use dependency locks (e.g. package-lock, pom.xml, requirements.txt) for stability.
- Check the status of external services.

7. Flaky Tests

- Flaky tests pass sometimes and fail at other times.
- Can be caused by timing issues, race conditions or unstable test data.
- Check if the same test failed in previous runs.
- Look for patterns (e.g. only on certain runners or times).
- Fix flaky tests instead of rerunning multiple times.

8. Evidence-Driven Investigation

- Collect and analyse evidence (logs, metrics, recent changes).
- Form a hypothesis and test it.
- Avoid making random changes or repeated reruns without understanding the cause.
- Document the root cause and the fix.
- Implement preventive measures to avoid recurrence.

Key Takeaways

- Identify the exact failing stage.
- Separate infrastructure failure from application failure.
- Check recent changes.
- Analyse runner evidence and build logs.
- Investigate dependency failures and flaky tests.
- Use evidence-driven investigation, not repeated reruns.
- Document the root cause and fix.
- Implement preventive measures.

Rollback, Roll Forward, and Release Recovery

When a release causes issues, you need a clear recovery strategy. This can be a rollback, a roll-forward fix, or a targeted mitigation. The right choice depends on the impact, risk, data changes and how quickly you can safely restore a healthy state.

1. When Rollback is Safe

- No database schema changes that break compatibility.
- The previous version is stable and known to work.
- Configuration changes can be reverted safely.
- No dependency changes that prevent the old version from running.
- Rollback can be completed quickly with minimal risk.

2. When Rollback is Dangerous

- Database schema changes are not backward compatible.
- Data has been modified or migrated.
- External dependencies have changed (APIs, services, contracts).
- The previous version may have security vulnerabilities.
- Rolling back could cause more downtime or data loss.

3. Roll-Forward Fixes

- Apply a new version with a fix.
- Use when rollback is risky or not possible.
- Fix configuration issues, code bugs or dependency problems.
- Often faster than a full rollback.
- Requires testing and verification before release.

4. Artifact Rollback

- Redeploy a previously known good artifact (image, package, build).
- Keep previous artifacts in your registry.
- Automate artifact rollback in your deployment pipeline.
- Ensure the old artifact is compatible with current infrastructure and dependencies.

5. Configuration Rollback

- Revert to the previous configuration (environment variables, config files, feature flags).
- Use version control for configuration (e.g. Git).
- Rollback infrastructure changes (e.g. Terraform, Helm, Kubernetes manifests).
- Validate the configuration before applying.

6. Database Compatibility

- Ensure the old application version can work with the current database schema.
- Use backward compatible migrations.
- Avoid breaking changes (dropping columns, changing data types).
- Consider expand-contract migrations.
- Test rollback and roll-forward with real data in staging before production.

7. Recovery Verification

- Confirm the service is healthy and working as expected.
- Check key metrics (error rate, latency, traffic).
- Verify logs and traces for remaining issues.
- Validate external dependencies.
- Monitor for a sufficient period to ensure stability.

8. Choosing the Smallest Safe Mitigation

- Assess the impact and blast radius.
- Choose the option with the lowest risk and fastest recovery.
- Consider rollback, roll-forward, configuration change or feature flag.
- Avoid making multiple changes at once.
- Always have a recovery plan before releasing.

Key Takeaways

- Not every issue requires a rollback.
- Rollback is safe only when there are no breaking changes.
- Roll-forward fixes are often the best option.
- Keep previous artifacts and configurations for quick recovery.
- Database changes can make rollback dangerous.
- Always verify the recovery before closing the incident.
- Choose the smallest safe mitigation to reduce risk.
- Plan recovery strategies before every release.

CI/CD Incident Response

A CI/CD incident can impact deployments, production services and the entire delivery process. A clear response plan helps you make the right decisions quickly, reduce risk, restore service and prevent the same issue from happening again.

1. Stop-the-Line Decisions



- Stop the pipeline when you detect critical issues (security, data loss, broken production).
- Prevent new deployments from reaching production.
- Notify the team immediately.
- Focus on stabilizing the current situation before resuming delivery.

2. Freeze Versus Continue Delivery

Freeze Delivery

- Use when the incident is severe or the root cause is unknown.
- Block new deployments to reduce risk.
- Allow only critical fixes.
- Keep the environment stable while investigating.

Continue Delivery

- Use when the incident is isolated and does not affect new changes.
- Continue with non-related pipelines if it is safe.
- Apply additional checks and approvals.
- Monitor closely for any impact on other services.

3. Identifying Affected Releases

- Find the exact release, commit or version that introduced the issue.
- Check recent deployments in the pipeline.
- Compare changes, configuration and dependency updates.
- Look at metrics, logs and traces to confirm the time of the impact.
- Identify if multiple services or environments are affected.

4. Containing Blast Radius

- Isolate the affected service or region.
- Use feature flags to disable new features.
- Block or reroute traffic if needed.
- Scale down or pause problematic components.
- Limit the impact to a small number of users or services.

5. Coordinating Production Changes

- Assign clear roles and responsibilities (e.g. incident commander, communicator, executor).
- Use a shared communication channel.
- Get approval before making production changes.
- Document all actions and decisions.
- Avoid multiple people making changes at the same time.

6. Restoring Service

- Apply the chosen mitigation (rollback, roll-forward, configuration change).
- Verify that the service is healthy.
- Check key metrics (error rate, latency, traffic, SLOs).
- Confirm that all dependencies are working.
- Monitor closely after recovery to ensure stability.

7. Preserving Evidence

- Keep pipeline logs, build logs and deployment logs.
- Save infrastructure and application logs.
- Capture relevant metrics and traces.
- Document the timeline of events.
- Preserve screenshots, alerts and communication records.
- This evidence is important for root cause analysis and future improvements.

8. Post-Incident Pipeline Improvements

- Identify the root cause and fix it.
- Add or improve automated checks (e.g. tests, security scans, validations).
- Improve monitoring and alerting.
- Update runbooks and documentation.
- Review and improve deployment and approval processes.
- Conduct a blameless post-incident review with the team.

Key Takeaways

- Make clear stop-the-line decisions when needed.
- Identify the affected releases quickly.
- Contain the blast radius to reduce impact.
- Coordinate and document all production changes.
- Restore service and verify stability.
- Preserve evidence for root cause analysis.
- Improve your CI/CD pipeline to prevent similar incidents.
- Always learn from the incident and make the system stronger.

Operating CI/CD at Platform Scale

As your organization grows, CI/CD needs to be scalable, consistent and easy to use. A well-designed platform enables multiple teams to deliver safely and independently without creating bottlenecks or losing control.

1. Reusable Workflows

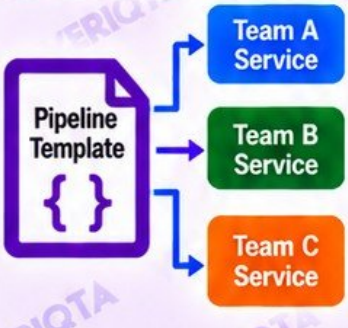


- Create reusable workflows for common tasks (build, test, scan, deploy).
- Store workflows in a central repository.
- Use versioning so teams can reliably use and update them.
- Reduce duplication and inconsistency across teams.

Example: Reusable Workflow (GitHub Actions)

```
jobs:
  build:
    uses: veriqta/platform-workflows/.github/workflows/build.yml@v1
    with:
      language: java
```

2. Pipeline Templates



- Provide standard pipeline templates for different application types.
- Include best practices (security, testing, artifact publishing).
- Allow teams to customize only what they need.
- Keep templates simple, documented and easy to use.

3. Golden Paths



- Define recommended ways to build, test and deploy.
- Provide opinionated, secure and maintainable defaults.
- Make the golden path easy and faster than doing it manually.
- Allow advanced teams to deviate when needed.

4. Self-Service Delivery



- Give teams the ability to deploy their services without waiting for the platform team.
- Provide clear documentation and examples.
- Automate approvals where possible.
- Use a developer portal or templates to simplify the process.

5. Policy as Code



- Enforce security, compliance and operational policies automatically.
- Use tools like OPA (Open Policy Agent), Conftest or cloud-native policy tools.
- Check policies in the pipeline (infrastructure and application code).
- Block or warn when policies are not met.

Example: OPA Policy (Requiring a Security Scan)

```
deny[msg] {
  input.pipeline.security_scan != true
  msg := "Security scan is required before deployment"
}
```

6. Central Standards versus Team Autonomy

Central Standards (Consistency)	Team Autonomy (Flexibility)
Security requirements	Application-specific steps
Base pipeline templates	Choice of tools (where allowed)
Deployment environments	Custom testing
Monitoring and logging	Release schedules
Compliance and auditing	Service configuration

Provide guardrails, not restrictions. Set mandatory standards for safety and compliance, but allow flexibility for innovation.

7. Multi-Team Ownership



- Define clear ownership of the CI/CD platform.
- Share responsibility across platform, security, SRE and development teams.
- Use codeowners and clear documentation.
- Create feedback loops with all teams.
- Align on SLAs and support processes.

8. Preventing the Platform Team from Becoming a Bottleneck



- Automate as much as possible.
- Provide self-service tools and templates.
- Keep the platform simple and well documented.
- Use a scalable support model (docs, chat, office hours).
- Measure usage and identify friction points.
- Continuously improve based on team feedback.

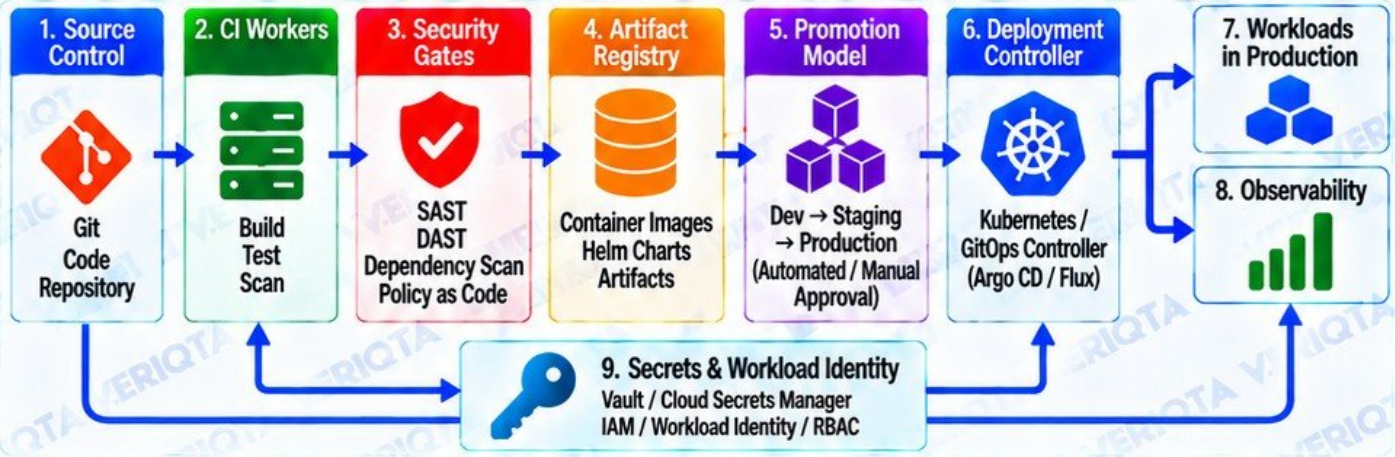
Key Takeaways

- Use reusable workflows and pipeline templates.
- Provide golden paths and self-service delivery.
- Enforce policies as code.
- Balance central standards with team autonomy.
- Enable multi-team ownership.
- Automate and document to avoid bottlenecks.
- A scalable CI/CD platform helps teams deliver safely, faster and more consistently.

Designing a Production-Grade Delivery Platform

A well-designed delivery platform enables teams to build, test, deploy and operate applications safely, consistently and at scale. It provides secure, automated, observable and reliable delivery from code to production.

1. Complete Reference Architecture



2. Source Control

- Use Git for version control.
- Branching strategy (main, develop, feature branches).
- Pull request reviews and approvals.
- Store infrastructure as code (IaC).
- Keep all application, pipeline and configuration code in Git.

3. CI Workers

- Use scalable runners (self-hosted or managed).
- Build the application.
- Run unit, integration and security tests.
- Build container images.
- Generate artifacts (binaries, images, Helm charts).

4. Security Gates

- SAST (static code analysis).
- DAST (dynamic analysis).
- Dependency scanning (open source and third-party).
- Policy as code (OPA, Conftest).
- Check for secrets, vulnerabilities and license issues.

5. Artifact Registry

- Store versioned container images, Helm charts and artifacts.
- Use private registries (ECR, GCR, ACR or Artifactory).
- Scan images for vulnerabilities.
- Apply retention and lifecycle policies.
- Use immutable tags (no :latest).

6. Promotion Model

- Promote the same artifact across environments.
- Use automated or manual approvals.
- Define clear promotion rules.
- Use environment-specific configuration.
- Track and audit all promotions.

7. Deployment Controller

- Use Kubernetes with GitOps (Argo CD or Flux).
- Automatically sync desired state from Git.
- Use deployment strategies (RollingUpdate, Canary, Blue/Green).
- Detect drift and self-heal.
- Support multi-cluster and multi-env deployments.

8. Secrets and Workload Identity

- Store secrets in a secure system (Vault, AWS Secrets Manager, etc).
- Use workload identity (IAM, OIDC).
- Avoid long-lived static credentials.
- Enable least privilege access (RBAC).
- Rotate secrets automatically.

9. Observability

- Collect metrics (Prometheus).
- Centralized logging (ELK, Loki).
- Distributed tracing (OpenTelemetry, Jaeger).
- Monitor pipeline and deployment health.
- Set alerts for failures and anomalies.

10. Progressive Delivery

- Use feature flags.
- Canary deployments.
- Blue/Green deployments.
- A/B testing.
- Gradual traffic shifting.
- Automated rollback on failure.

11. Rollback and Recovery

- Support quick rollback to previous version.
- Automate rollback in the deployment controller.
- Validate service health after rollback.
- Have runbooks for manual recovery.
- Keep database and configuration compatibility in mind.

12. Final Staff Engineer Architecture Decisions

- Choose the right tools for your organization.
- Design for scalability, security and reliability.
- Balance standardization with team flexibility.
- Enable self-service without losing control.
- Plan for multi-tenancy and multi-environment.
- Ensure the platform is observable, maintainable and cost effective.

Key Takeaways

- Use a complete and secure reference architecture.
- Automate testing, security and deployments.
- Promote the same artifact across environments.
- Provide self-service with guardrails.
- Monitor everything (pipelines, deployments and services).
- Support progressive delivery and quick recovery.
- Keep secrets safe and use workload identity.
- Make architectural decisions that scale with your organization.